VERA RBL-XE
FPGA Build & Programming Guide

iCE40UP5K – Rev 2

6502 bus controller with RAMbo 256 KB, PBI RAM/ROM,
and external SRAM support

Last updated: June 11, 2026

0 Contents

1 Overview

This document describes the complete workflow for building and programming the FPGA firmware for the VERA RBL-XE module. The firmware implements a 6502 bus controller on a Lattice iCE40UP5K, replacing the ESP32-S3 MCU used in the previous design.

1.1 Architecture summary

- **PBI RAM** (`$D600`–`$D7FF`, 512 B): stored in iCE40 BRAM (1 EBR block).
- **PBI ROM** (`$D800`–`$DFFF`, 2 KB): stored in iCE40 BRAM (4 EBR blocks), pre-loaded at synthesis time from the assembled 6502 driver binary.
- **RAMbo** (`$4000`–`$7FFF`, 16 KB window, 256 KB total): served by an external 512K×8 SRAM (e.g. AS6C4008-55SIN). The FPGA controls chip-enable, output-enable, write-enable, and the upper four address bits (bank), using only 7 new I/O pins.
- **PHI2 synchronisation**: PHI2 is treated as a data input and synchronised to the 25 MHz FPGA clock via a two-stage flip-flop chain, producing a single-cycle `phi2_rise` pulse used to trigger all registered logic.

1.2 Memory map

Range	Size	Function	Implementation
`$4000`–`$7FFF`	16 KB window	RAMbo (256 KB banked)	External SRAM
`$D1FF`	1 B	VERA_CS latch (write)	Register
`$D301`	1 B	PIA PORTB snoop	Register
`$D303`	1 B	PIA PBCTL snoop	Register
`$D600`–`$D7FF`	512 B	PBI RAM	BRAM (1 EBR)
`$D800`–`$DFFF`	2 KB	PBI ROM	BRAM (4 EBR)

2 Prerequisites

2.1 FPGA toolchain

All tools below are open-source. Install on Debian/Ubuntu:

```
sudo apt install fpga-icestorm nextpnr-ice40 yosys ghdl
```

Note: the `ghdl-yosys-plugin` is required for VHDL synthesis and is generally not available as a package. Build it from source:

```
git clone https://github.com/ghdl/ghdl-yosys-plugin
cd ghdl-yosys-plugin
make
sudo make install
```

2.2 Flash programming tools

```
# openFPGALoader -- recommended, supports many programmers
sudo apt install openFPGALoader

# iceprog is already included in fpga-icestorm
```

2.3 6502 assembler (cc65)

The PBI ROM driver is written in 6502 assembly and built with the cc65 toolchain.

```
sudo apt install cc65
```

2.4 Python 3

Required to run `gen_pbi_rom_pkg.py`, which converts the assembled 6502 binary into a VHDL constant package.

```
sudo apt install python3
```

2.5 Tool roles at a glance

Tool	Input	Output	Purpose
`ca65` / `ld65`	`.s` sources	`.bin`	6502 assembly
`gen_pbi_rom_pkg.py`	`.bin`	`.vhd`	ROM initialisation
`ghdl` (plugin)	VHDL sources	RTLIL	VHDL elaboration
`yosys`	RTLIL	`.json`	Synthesis
`nextpnr-ice40`	`.json + .pcf`	`.asc`	Place & route
`icepack`	`.asc`	`.bin`	Bitstream packing
`openFPGALoader`	`.bin`	SPI flash	Programming

3 Build Flow

3.1 Step 1 – Assemble the 6502 ROM driver

The PBI ROM at `$D800`–`$DFFF` contains a 2 KB 6502 driver assembled from `firmware/MCU/6502/src/vera_pbi_`

```
cd firmware/MCU/6502
make
```

Listing 1: Assemble the 6502 driver

This single command runs three sub-steps:

1. **Assemble**: `ca65` compiles `vera_pbi_handler.s` into `vera_pbi_handler.o`.
2. **Link**: `ld65` links with `pbi-driver.ld` to produce `vera_pbi_handler.bin`.
3. **Generate VHDL ROM package**: `gen_pbi_rom_pkg.py` reads the binary and writes `../../FPGA/pbi_rom_pkg.vhd` with the `PBI_ROM_INIT` constant.

To regenerate only the VHDL package when the binary already exists:

```
make fpga-rom
```

3.2 Step 2 – Synthesis

Synthesis converts the VHDL sources into a technology-mapped netlist for the iCE40 family.

```
cd firmware/FPGA
yosys -m ghdl -p \
    "ghdl --std=08 custom_types.vhd pbi_rom_pkg.vhd C6502_Interface.
        vhd \
```

```
        -e C6502_Interface; \
    synth_ice40 -top C6502_Interface -json C6502_Interface.json"
```

Listing 2: Synthesis with Yosys + GHDL plugin

The `-m ghdl` flag loads the GHDL plugin into Yosys, which handles VHDL analysis and elaboration. The `synth_ice40` pass maps the resulting netlist to iCE40 primitives (LUTs, FFs, BRAM, SB_IO, etc.) and writes a JSON netlist.

On success, Yosys prints a resource utilisation summary. Expected ranges:

Resource	Expected usage
SB_LUT4 (logic cells)	80–180
SB_FF (flip-flops)	30–60
SB_RAM40_4K (EBR)	5
SB_IO (I/O buffers)	40

3.3 Step 3 – Place & Route

Place & route assigns logic cells and routes signals to physical iCE40 resources, guided by the pin constraints file.

```
nextpnr-ice40 --up5k --package sg48 \
    --json C6502_Interface.json \
    --pcf board-pin-mapping.pcf \
    --asc C6502_Interface.asc
```

Listing 3: Place & route with nextpnr-ice40

Important: all pin numbers in `board-pin-mapping.pcf` are placeholders. Edit the file to match your actual schematic before this step (see Section 5).

nextpnr reports timing after routing. The design must meet timing at 25 MHz (the system clock). The critical path is the PHI2 synchroniser chain, which is combinational; the path from `phi2_rise` to any registered output is unconstrained by PHI2 frequency.

3.4 Step 4 – Pack the bitstream

`icepack` converts the ASCII intermediate format produced by nextpnr into the binary bitstream that is written to the SPI flash.

```
icepack C6502_Interface.asc C6502_Interface.bin
```

Listing 4: Pack binary bitstream

`C6502_Interface.bin` is the file that gets programmed into the SPI flash. Its size is typically around 100–200 KB depending on the device configuration.

3.5 Step 5 – Program the SPI flash

The iCE40UP5K loads its configuration from an external SPI NOR flash at power-up. The programmer writes the bitstream into that flash.

3.5.1 Using openFPGALoader (recommended)

```
openFPGALoader -b ice40_generic C6502_Interface.bin
```

Listing 5: Program with openFPGALoader

Select the programmer cable explicitly if needed:

```
openFPGALoader --cable ft2232    C6502_Interface.bin   # FTDI FT2232H
openFPGALoader --cable ft232     C6502_Interface.bin   # FTDI FT232H
openFPGALoader --cable tigard    C6502_Interface.bin   # Tigard
openFPGALoader --cable raspberrypi C6502_Interface.bin # Raspberry Pi
   GPIO
```

3.5.2 Using iceprog (FTDI FT2232H)

```
iceprog C6502_Interface.bin

# Erase + write (useful if previous write was partial)
iceprog -e 1048576 C6502_Interface.bin
```

Listing 6: Program with iceprog

3.6 One-command build and flash

The `firmware/FPGA/Makefile` wraps all steps:

```
cd firmware/FPGA
make              # synthesise + P&R + pack
make flash        # write with openFPGALoader
make flash-iceprog   # write with iceprog
make clean        # remove generated files
```

4 Programmer Wiring

The SPI flash is shared between the FPGA (configuration bus at boot) and the programmer (writes the bitstream). During programming, the FPGA is held in reset by asserting `CRESET_B` low.

Programmer signal	iCE40UP5K pin	SPI flash pin
MOSI	SPI_SI	DI
MISO	SPI_SO	DO
SCK	SPI_SCK	CLK
CS#	–	CS#
CRST#	CRESET_B	–
CDONE	CDONE	–

Recommended SPI flash: Winbond W25Q16 (2 MB) or W25Q32 (4 MB).

5 Pin Assignment

Edit `board-pin-mapping.pcf` and replace every pin number with the actual pad number from your schematic before running nextpnr-ice40.

The `CLK` signal must be assigned to a **Global Buffer (GB) input** pin. Valid GB pins on the iCE40UP5K SG48 package are: **20, 28, 31, 38, 41, 43, 48**.

5.1 Complete pin table

Signal	Dir	PCF pin	Std	Description
`CLK`	In	35*	LVCMOS33	25 MHz system clock. Must land on a GB pin.
`A[0]`	In	40*	LVCMOS33	6502 address bit 0
`A[1]`	In	41*	LVCMOS33	6502 address bit 1
`A[2]`	In	42*	LVCMOS33	6502 address bit 2
`A[3]`	In	43*	LVCMOS33	6502 address bit 3
`A[4]`	In	44*	LVCMOS33	6502 address bit 4
`A[5]`	In	45*	LVCMOS33	6502 address bit 5
`A[6]`	In	46*	LVCMOS33	6502 address bit 6
`A[7]`	In	47*	LVCMOS33	6502 address bit 7
`A[8]`	In	48*	LVCMOS33	6502 address bit 8
`A[9]`	In	1*	LVCMOS33	6502 address bit 9
`A[10]`	In	2*	LVCMOS33	6502 address bit 10
`A[11]`	In	3*	LVCMOS33	6502 address bit 11
`A[12]`	In	4*	LVCMOS33	6502 address bit 12
`A[13]`	In	7*	LVCMOS33	6502 address bit 13. Also wired to SRAM A[13] on PCB.
`A[14]`	In	9*	LVCMOS33	6502 address bit 14
`A[15]`	In	10*	LVCMOS33	6502 address bit 15
`D[0]`	InOut	11*	LVCMOS33	6502 data bit 0. Shared net with SRAM DQ[0].
`D[1]`	InOut	12*	LVCMOS33	6502 data bit 1. Shared net with SRAM DQ[1].
`D[2]`	InOut	13*	LVCMOS33	6502 data bit 2. Shared net with SRAM DQ[2].
`D[3]`	InOut	14*	LVCMOS33	6502 data bit 3. Shared net with SRAM DQ[3].
`D[4]`	InOut	15*	LVCMOS33	6502 data bit 4. Shared net with SRAM DQ[4].
`D[5]`	InOut	16*	LVCMOS33	6502 data bit 5. Shared net with SRAM DQ[5].
`D[6]`	InOut	17*	LVCMOS33	6502 data bit 6. Shared net with SRAM DQ[6].
`D[7]`	InOut	18*	LVCMOS33	6502 data bit 7. Shared net with SRAM DQ[7].
`PHI2`	In	34*	LVCMOS33	6502 phase-2 clock. Synchronised internally; not used as FPGA clock.
`RNW_`	In	36*	LVCMOS33	Read/Not-Write. High = read, Low = write.
`VERA_CS`	Out	32*	LVCMOS33	VERA chip-select latch output (active low).
`MPD`	Out	31*	LVCMOS33	Machine Program Data. Asserted low for `$D800`–`$DFFF` reads; blocks Atari from driving D[7:0].
`EXTSEL`	Out	30*	LVCMOS33	External Select. Asserted low for PBI RAM (`$D600`–`$D7FF`) and RAMbo (`$4000`–`$7FFF`) accesses; blocks Atari internal RAM from responding.

continued on next page

Signal	Dir	PCF pin	Std	Description
`DIP_SEL[0]`	In	27*	LVCMOS33	DIP switch bit 0. Selects which D[n] bit controls VERA_CS latch.
`DIP_SEL[1]`	In	28*	LVCMOS33	DIP switch bit 1.
`DIP_SEL[2]`	In	29*	LVCMOS33	DIP switch bit 2.
`SRAM_A_BANK[0]`	Out	19*	LVCMOS33	SRAM address bit 14 (bank LSB).
`SRAM_A_BANK[1]`	Out	20*	LVCMOS33	SRAM address bit 15.
`SRAM_A_BANK[2]`	Out	21*	LVCMOS33	SRAM address bit 16.
`SRAM_A_BANK[3]`	Out	22*	LVCMOS33	SRAM address bit 17 (bank MSB).
`SRAM_CE_N`	Out	23*	LVCMOS33	SRAM chip enable (active low). Asserted during PHI2-high when address is in `$4000`–`$7FFF` and RAMbo is enabled.
`SRAM_OE_N`	Out	24*	LVCMOS33	SRAM output enable (active low). Asserted on read cycles only.
`SRAM_WE_N`	Out	25*	LVCMOS33	SRAM write enable (active low). Asserted on write cycles only. Pulse width equals PHI2-high time (≈279 ns at 1.79 MHz).

** Pin numbers are placeholders. Replace with actual pad numbers from your board schematic.*

5.2 External SRAM address wiring

The SRAM address bus is split between PCB traces (lower 14 bits) and FPGA outputs (upper 4 bank bits):

SRAM address bit	Source	Note
A[13:0]	6502 A[13:0] (PCB net)	Direct trace, no FPGA I/O pin
A[14]	FPGA `SRAM_A_BANK[0]`	Bank bit 0 = PORTB[2]
A[15]	FPGA `SRAM_A_BANK[1]`	Bank bit 1 = PORTB[3]
A[16]	FPGA `SRAM_A_BANK[2]`	Bank bit 2 = PORTB[5]
A[17]	FPGA `SRAM_A_BANK[3]`	Bank bit 3 = PORTB[6]
A[18]	GND (PCB)	Selects lower 256 KB of 512 KB chip

5.3 RAMbo bank formula

The FPGA snoops writes to the Atari PIA to derive the active bank:

```
PORTB at $D301  -- bank bits
PBCTL at $D303  -- bit 2 enables RAMbo

bank[1:0] = PORTB[3:2]
bank[3:2] = PORTB[6:5]
=> SRAM A[17:14] = { PORTB[6], PORTB[5], PORTB[3], PORTB[2] }
```

RAMbo is active only when `PBCTL[2] = 1`.

6 Timing

6.1 Read cycle (SRAM)

Event	Time from PHI2 rise	Note
Address valid (6502 tADS)	-40 ns	Before PHI2 rise
SRAM_CE# asserted	≈ 0 ns	Combinational gate on PHI2
SRAM data valid (tAA=55 ns)	$+55$ ns	AS6C4008-55
6502 data setup deadline	$+249$ ns	279 ns $-$ 30 ns tDSR
Read margin	**+194 ns**	Adequate

6.2 Write cycle (SRAM)

WE# is asserted combinationally when PHI2 is high and the address is in the RAMbo window. The pulse width equals the PHI2-high time:

$$t_{WC} = t_{\text{PHI2-high}} \approx 279\,\text{ns} \quad \gg \quad t_{WC(\text{SRAM})} = 55\,\text{ns}$$

6.3 PBI RAM / ROM (BRAM)

BRAM reads are triggered on `phi2_rise` (one clock cycle after the real PHI2 edge, ≈40 ns delay). The BRAM output is registered and valid within one additional clock cycle (≈40 ns), leaving ≈199 ns of margin before the 6502 samples the bus.

7 Source File Reference

File	Description
`C6502_Interface.vhd`	Top-level VHDL entity (Rev 2). Contains all bus logic, BRAM instances, and SRAM control.
`custom_types.vhd`	Package defining `rom_array` (2048×8) and `ram_512_array` (512×8) array types.
`pbi_rom_pkg.vhd`	Auto-generated package containing the `PBI_ROM_INIT` constant (2048 bytes). Do not edit manually.
`gen_pbi_rom_pkg.py`	Python 3 script that converts a 6502 `.bin` file into `pbi_rom_pkg.vhd`. Invoked by the MCU Makefile.
`board-pin-mapping.pcf`	Pin constraints for iCE40UP5K SG48. All pin numbers are placeholders – edit before use.
`Makefile`	Targets: `all`, `flash`, `flash-iceprog`, `clean`.